

MAIE 5532: Machine Learning Systems

Week 6 – Hardware Backend Specialization and Optimizations

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Week 6 Overview

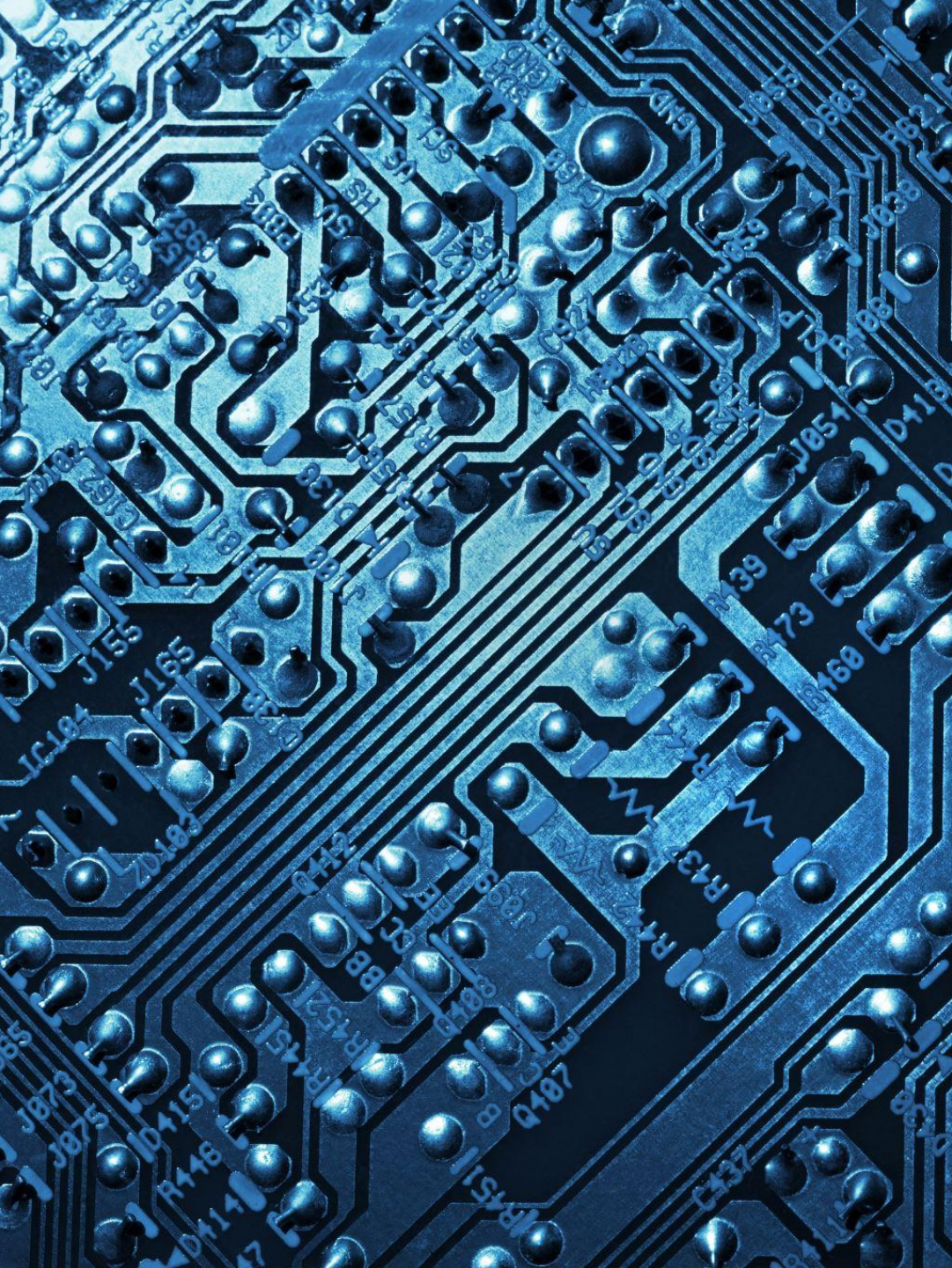
Hardware Backend Specialization: From Data Centers to Microcontrollers

Learning Objectives:

- Understand the hardware spectrum and specialization principles for AI acceleration
- Analyze architectural trade-offs from general-purpose to domain-specific processors
- Design hardware-aware models for diverse deployment targets
- Evaluate energy, performance, and cost implications across the hardware continuum

Required Readings:

- [Book 1 - Volume 1 — Hardware Acceleration](#)
- [Book 1 - Volume 1 — Model Compression](#)
- [Book 1 - Volume 2 — Edge Intelligence](#)
- [Book 2: Chapter 2: Getting Started \(hardware overview\)](#)
- [Book 2: Chapter 14: Designing Your Own TinyML Applications](#)



The Hardware Specialization Spectrum

Why Specialized Hardware for AI?

The Challenge:

General-purpose processors face fundamental limitations when executing AI workloads:

- Von Neumann bottleneck: Data transfer between processor and memory
- Instruction overhead: Complex decode and dispatch for simple operations
- Energy inefficiency: General circuits consume more power than specialized ones

The Evolution:

1980s: Intel 8087 floating-point coprocessor (100x speedup for arithmetic)

1990s: Graphics processing units for parallel rendering

2000s: Digital signal processors for audio/video

2010s: Tensor processing units for machine learning

Real-world Impact:

Modern smartphones decode 4K video at 60fps while consuming only 2-3 watts through dedicated video codecs - a task that would overwhelm general-purpose processors.

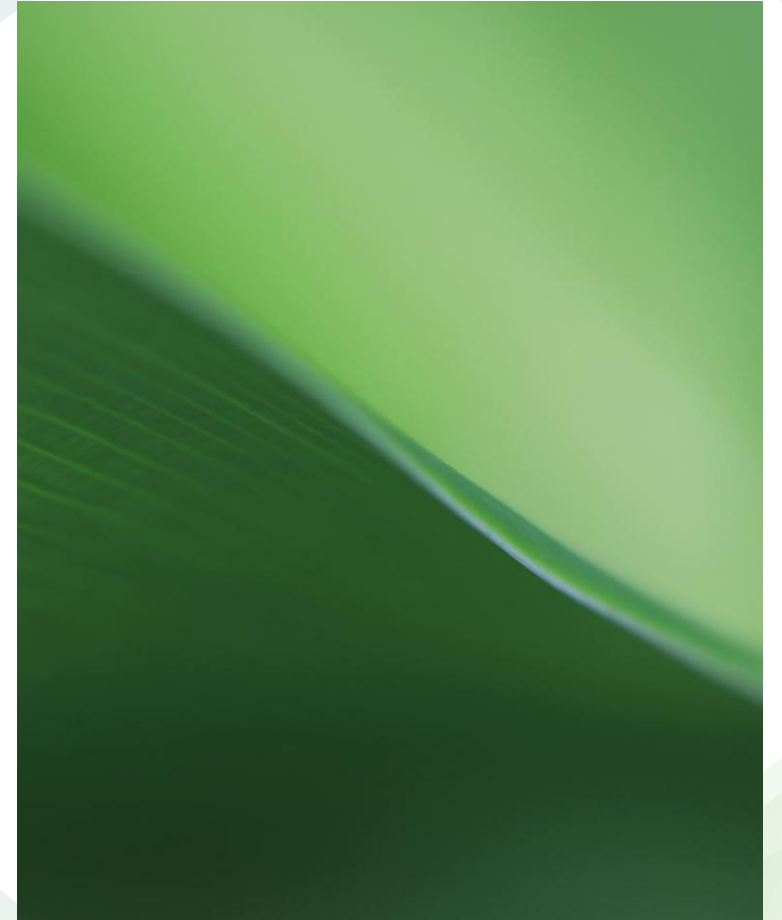
Session Structure

SESSION 1: HARDWARE ARCHITECTURE FUNDAMENTALS

- EVOLUTION FROM GENERAL-PURPOSE TO DOMAIN-SPECIFIC ARCHITECTURES
- MEMORY HIERARCHY AND ENERGY EFFICIENCY
- SPECIALIZED PROCESSORS ANALYSIS
- GPU, TPU, DESIGN, AND MOBILE AI ACCELERATORS

SESSION 2: HARDWARE-AWARE DESIGN

- MODEL ARCHITECTURE DECISIONS FOR DIFFERENT HARDWARE TARGETS
- NUMERICAL PRECISION AND QUANTIZATION STRATEGIES
- MEMORY BANDWIDTH OPTIMIZATION TECHNIQUES
- DESIGN METHODOLOGIES FOR HARDWARE-SOFTWARE CO-OPTIMIZATION



SESSION 1: HARDWARE ARCHITECTURE FUNDAMENTALS

- The Limits of General-Purpose Computing
- Understanding the Motivation for Specialization
- CPU Limitations for AI Workloads:

```
// Typical neural network computation on CPU
for (int n = 0; n < batch_size; n++) {
  for (int out = 0; out < output_neurons; out++) {
    float sum = bias[out];
    for (int in = 0; in < input_neurons; in++) {
      sum += input[n][in] * weights[out][in]; // One
operation per cycle
    }
    output[n][out] = relu(sum); // Branch introduces
pipeline stalls
  }
}
```

Inefficiencies:

- Sequential execution: One multiply-accumulate per cycle
- Branch overhead: Conditional operations stall instruction pipeline
- Memory bottleneck: Frequent loads from slow main memory
- Instruction overhead: Complex decode for simple arithmetic

The Opportunity:

AI workloads exhibit predictable patterns ideal for hardware acceleration through parallelism, specialized datapaths, and optimized memory hierarchies.

Domain-Specific Architecture Principles

Design Philosophy for AI Acceleration

Core Specialization Strategies

1. Customized Datapaths

- Matrix multiplication units optimized for dense linear algebra
- Systolic arrays for rhythmic data flow
- Dedicated activation function units

2. Specialized Memory Hierarchies

- Large on-chip buffers for weight and activation reuse
- High-bandwidth memory interfaces
- Software-managed scratchpad memory

3. Reduced Instruction Overhead

- Domain-specific instruction sets
- Single instructions for complex operations
- Hardware-managed execution pipelines

4. Direct Hardware Implementation

- Fixed-function blocks for common operations
- Elimination of software interpretation overhead

AI Compute Primitives Foundation

- **Building Blocks of AI Acceleration**
- Vector Operations - SIMD Execution:

```
// Traditional scalar processing
for (i = 0; i < 1024; i++) {
    result[i] = input[i] * weight[i]; // One operation per cycle
}

// Vector processing (8-wide SIMD)
for (i = 0; i < 1024; i += 8) {
    vec_result = vec_multiply(vec_input, vec_weight); // 8
    operations per cycle
}
```

- **Matrix Operations - Structured Parallelism:**
Neural network layers perform matrix multiplication $C = A \times B$ where specialized hardware can process entire blocks simultaneously rather than individual elements.
- **Special Function Units:**
Dedicated hardware for activations (ReLU, sigmoid), normalization (batch norm), and mathematical functions (square root, exponential) eliminates complex software implementations.

Memory Hierarchy Design for AI

Addressing the AI Memory Wall

The Memory Challenge:

- AI workloads require massive data movement:
- Model parameters: Billions of weights to store and access
- Intermediate activations: Large tensors between layers
- High bandwidth demands: Modern models need TB/s memory bandwidth

Memory access pattern in neural networks

- Weights: Read once per input batch
- Activations: Write once, read multiple times
- Gradients (training): Write once per parameter update

Hierarchical Solution:

- **Registers:** Immediate operands (< 1 cycle latency)
- **L1/L2 Cache:** Recently used weights and activations (1-10 cycles)
- **Scratchpad Memory:** Software-managed intermediate storage (5-20 cycles)
- **High-Bandwidth Memory:** Large parameter storage (100 cycles)
- **Off-chip DRAM:** Full model storage when too large for on-chip memory

Design Principle: Keep frequently accessed data as close to compute units as possible.

Energy Efficiency in AI Hardware

Power Consumption Across the Computing Stack

Energy Cost Hierarchy:

- 32-bit floating-point addition: ~20 femtojoules
- On-chip SRAM access: ~5 picojoules
- Off-chip DRAM access: ~1.3 nanojoules (65,000x more than computation!)

femto-: 10^{-15} (one quadrillionth)

pico-: 10^{-12} (one trillionth)

nano-: 10^{-9} (one billionth)

Optimization Strategies:

1. Minimize Data Movement

- Maximize on-chip computation before external memory access
- Reuse data across multiple operations

2. Precision Reduction

- 8-bit integer operations consume ~5x less energy than 32-bit floating point
- Specialized low-precision arithmetic units

3. Specialized Circuits

- Fixed-function units more energy-efficient than programmable logic
- Elimination of instruction fetch and decode overhead

4. Voltage and Frequency Scaling

- Lower voltage operation for energy-constrained environments
- Dynamic frequency scaling based on workload requirements

Numerical Precision in AI Hardware

Trading Accuracy for Efficiency

Precision Format Evolution:

```
// Training: High precision for gradient stability
float32 weights[1024][512]; // 32-bit floating point
float32 activations[batch][1024];

// Inference: Reduced precision for efficiency
int8_t quantized_weights[1024][512]; // 8-bit integer
int8_t quantized_activations[batch][1024];

// Ultra-low power: Extreme quantization
int4_t binary_weights[1024][512]; // 4-bit or binary
```

Hardware Implications:

- **FP32**: Full accuracy, high energy cost, large storage
- **FP16/BF16**: 2x memory reduction, faster computation on modern hardware
- **INT8**: 4x memory reduction, specialized integer units, minimal accuracy loss
- **INT4/Binary**: Extreme efficiency, requires careful algorithm design

Mixed-Precision Computing:

Modern accelerators support different precisions for different operations:

- Matrix multiplication: INT8 or FP16
- Accumulation: FP32 for numerical stability
- Activation functions: INT8 for efficiency

SPECIALIZED PROCESSORS ANALYSIS



CUDA, which stands for Compute Unified Device Architecture, is a parallel computing platform and programming model created by NVIDIA for its graphics processing units (GPUs). It allows software to use the GPU for general-purpose computing to significantly speed up tasks in areas like AI, scientific computing, and graphics.

GPU Architecture for AI Acceleration

From Graphics to General-Purpose AI Computing

SIMT (Single Instruction, Multiple Threads) Model:

```
// CUDA kernel executing on thousands of threads
__global__ void matrix_multiply(float* C, float* A, float* B, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float sum = 0.0f;
    for (int k = 0; k < N; k++) {
        sum += A[row * N + k] * B[k * N + col]; // Parallel across threads
    }
    C[row * N + col] = sum;
}
```

Architectural Features:

- **Streaming Multiprocessors:** 80+ independent processing units
- **Tensor Cores:** Dedicated 4x4 matrix multiplication units
- **High-Bandwidth Memory:** Up to 2 TB/s memory bandwidth
- **Hierarchical Memory:** Shared memory, L2 cache, global memory

GPU Advantages for AI:

- **Massive parallelism:** Thousands of threads executing simultaneously
- **Mature software ecosystem:** CUDA, OpenCL, extensive libraries
- **Programmability:** Flexible execution model for diverse algorithms

Tensor Processing Units (TPUs)

Google's Specialized AI Architecture

Systolic Array Design:

TPUs implement large 2D arrays of processing elements where data flows rhythmically through the grid:

```
// Conceptual systolic array operation
for (int cycle = 0; cycle < matrix_size; cycle++) {
// Each PE performs: accumulator += input_from_top * weight_from_left
// Data flows: inputs downward, weights rightward, outputs collected
PE[i][j].accumulate(input_from_above[i], weight_from_left[j]);

// Shift data for next cycle
shift_inputs_down();
shift_weights_right();
}
```

TPU Characteristics:

- **Large Matrix Units:** 128×128 or 256×256 systolic arrays
- **Mixed Precision:** BF16 computation with FP32 accumulation
- **Software Stack:** XLA compiler for automatic optimization
- **High Throughput:** Optimized for training large models

Design Trade-offs:

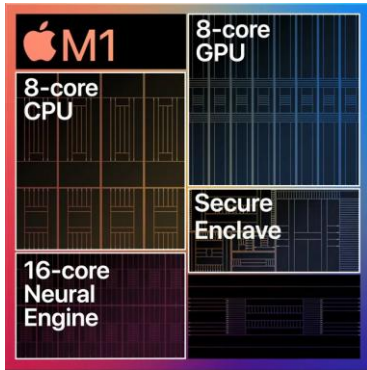
- Excellent for large matrix operations
- Less flexible than GPUs for irregular computations
- Requires compiler optimization for efficiency



Google's Tensor Processing Unit (TPU)
Image Source: unwire.pro

Mobile AI Accelerators

Neural Processing Units for Edge Devices



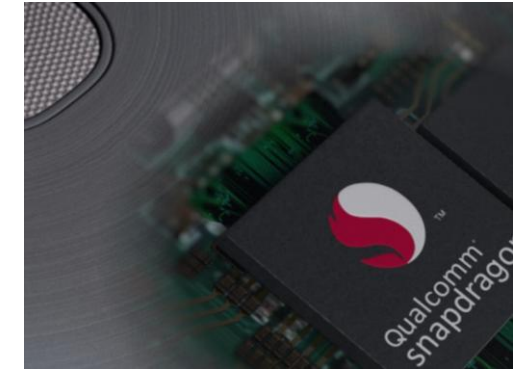
Apple Neural Engine Architecture

- 16 cores optimized for neural network inference
- 15.8 TOPS at INT8 precision
- Integrated with CPU and GPU for heterogeneous computing



ARM Mali GPU with ML Extensions

- Dedicated ML processing units alongside graphics cores
- Mixed-precision support: FP16, INT8, INT4
- **Power-efficient execution for mobile battery life**



Qualcomm Hexagon DSP

- Vector processing optimized for signal processing and ML
- Dedicated Tensor Accelerator for neural network operations
- Integration with camera and audio subsystems

Mobile Design Constraints:

- **Power Budget:** < 5W total system power including display
- **Thermal Limits:** Passive cooling in thin form factors
- **Memory Bandwidth:** Limited compared to datacenter hardware
- **Cost Sensitivity:** Consumer price points drive design decisions

Edge and Microcontroller AI

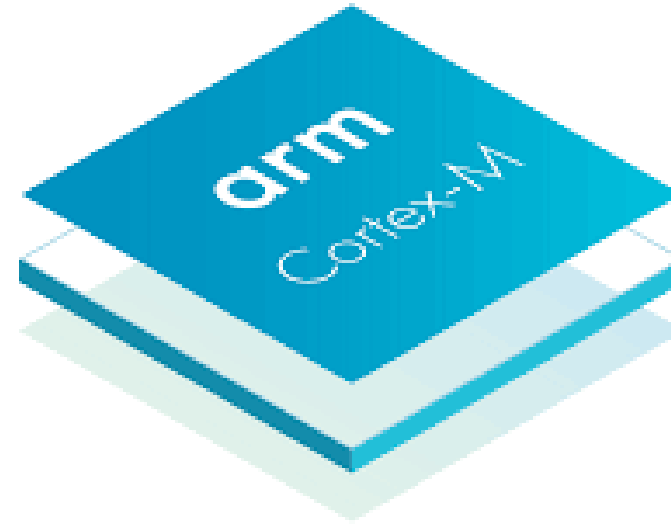
Ultra-Low Power AI Processing

Microcontroller Characteristics:

```
// Typical microcontroller AI inference
void run_inference() {
    // Model stored in flash memory (read-only)
    const int8_t* weights = (int8_t*)FLASH_MODEL_BASE;

    // Activations in SRAM (limited to ~256KB)
    int8_t activations[MAX_ACTIVATION_SIZE];

    // Quantized computation
    for (int layer = 0; layer < num_layers; layer++) {
        quantized_conv2d(&activations[input_offset],
                        &weights[weight_offset],
                        &activations[output_offset]);
    }
}
```



Hardware Constraints:

- **Memory:** 32KB - 1MB SRAM, 256KB - 2MB Flash
- **Computation:** 100 MHz ARM Cortex-M processors
- **Power:** 1-100 mW operation, battery life measured in months
- **Cost:** \$1-10 per processor in volume

Optimization Strategies:

- **8-bit quantization:** Essential for memory and computation efficiency
- **Model compression:** Pruning, knowledge distillation
- **Streaming computation:** Process data without storing full tensors
- **Hardware acceleration:** ARM CMSIS-NN optimized kernels

In-Class Exercise 1 (20 minutes)

Hardware Selection and Optimization

Scenario: Smart Security Camera System

You're designing an on-device smart security camera for outdoor use with privacy constraints (no continuous cloud inference). Pick your own targets, then justify a design that meets them.

Step 1 — Choose your targets (pick one from each)

- Budget: \$50, \$80, or \$120 total system cost
- Battery profile:
 - Continuous: 3 days, 1 week, or 1 month
 - Or Motion-triggered: 10% duty cycle (assume PIR or similar)
- FPS requirement: 10, 20, or 30 fps
- Deployment scene: doorway (narrow FOV) or parking lot (wide FOV)

Step 2 — Pick Hardware



ESP32-S3 + Camera: \$15,
~0.5 W active, ~0.05 W idle



Qualcomm QCS605 dev board: \$70–\$100, ~3–4 W active, ~0.5 W deep idle



Raspberry Pi 4 + Intel NCS2: \$75, ~4 W active idle ~3 W



NVIDIA Jetson Nano: \$99, ~10 W active (5 W with performance cap)

Your task (groups of 3-4)

1. State your chosen targets (budget, battery profile, FPS, scene).
2. Select hardware and justify using power, cost, and capability.
3. Choose model and optimizations: architecture, input resolution, quantization, and any pruning/distillation.
4. Estimate battery life vs. power:
5. Compute average power with your duty cycle. Show that it meets your chosen battery target using $Wh = V \times Ah$.
6. Describe system tactics: duty-cycling, motion trigger, frame skipping, batching, thermal considerations.
7. Identify trade-offs you accept (accuracy, resolution, latency, detection range).

- Optional sensors: PIR motion sensor \$3, microSD \$5, LTE/Wi-Fi module \$10–\$20 (for alerts only, no cloud inference)
- Battery reference: 10,000 mAh at 3.7 V $\approx$ 37 Wh

Systolic Array Deep Dive

Understanding Specialized Matrix Computation

Systolic Array Principles:

Data flows through a 2D grid of processing elements in a synchronized, rhythmic pattern

Advantages:

- **High Utilization:** All PEs active simultaneously during steady state
- **Predictable Data Flow:** No complex routing or scheduling
- **Scalability:** Linear scaling with array size
- **Energy Efficiency:** Minimal control overhead

Limitations:

- **Matrix Size Constraints:** Optimal for specific dimension ranges
- **Setup Overhead:** Filling pipeline takes time
- **Limited Flexibility:** Primarily optimized for dense matrix operations

Applications:

- Convolutional layers (after im2col transformation)
- Fully connected layers
- Attention mechanisms in transformers

Reminder from Week 3: Systolic Arrays use Tiling

```
// PE (Processing Element) operation each cycle
class ProcessingElement {
    int32_t accumulator;

    void cycle(int8_t input_from_north, int8_t weight_from_west) {
        // Multiply-accumulate operation
        accumulator += input_from_north * weight_from_west;

        // Pass data to neighbors
        send_to_south(input_from_north);
        send_to_east(weight_from_west);
    }
};
```

Memory Bandwidth Optimization

Feeding the Compute Beast

The Bandwidth Challenge:

```
// Memory bandwidth calculation for inference
// Model: 100M parameters × 4 bytes/param = 400MB weights
// Inference rate: 100 inferences/second
// Required bandwidth: 400MB × 100 = 40 GB/s just for weights!
```

```
// Add activations and you easily exceed available bandwidth
```

Implementation Example:

```
// Tiled computation to maximize cache reuse
void optimized_conv2d() {
    for (int tile_y = 0; tile_y < height; tile_y += TILE_SIZE) {
        for (int tile_x = 0; tile_x < width; tile_x += TILE_SIZE) {
            // Process tile completely before moving to next
            // Maximizes reuse of loaded weights and inputs
            process_tile(tile_x, tile_y);
        }
    }
}
```

Optimization Strategies:

Weight Reuse

- Batch processing: Amortize weight reads across multiple inputs
- Temporal reuse: Keep weights in cache across operations

Activation Reuse

- Spatial reuse: Overlap in convolutional operations
- Channel reuse: Same spatial locations across feature maps

Precision Reduction

- 8-bit weights: 4x bandwidth reduction
- Mixed precision: Reduce precision where accuracy permits

Compression

- Sparse matrices: Store only non-zero values
- Huffman coding: Compress weight distributions

SESSION 2: HARDWARE- AWARE DESIGN





Architecture Selection Principles

For GPUs (Parallel Throughput)

- Favor wide layers and large batch sizes to maximize tensor core/SM occupancy and enable kernel fusion.
 - Use regular operators that compile and fuse well: predominantly 3×3 and 1×1 convolutions, simple activations, and standard normalization.
 - Keep compute dense and contiguous; avoid tiny fully connected tails or rare ops that break fusion and create launch overhead.
 - Ensure tensors align with preferred tile sizes and memory layouts (e.g., channels multiples that match vector widths).
-



Architecture Selection Principles

For Mobile/Edge SoCs (Efficiency-First)

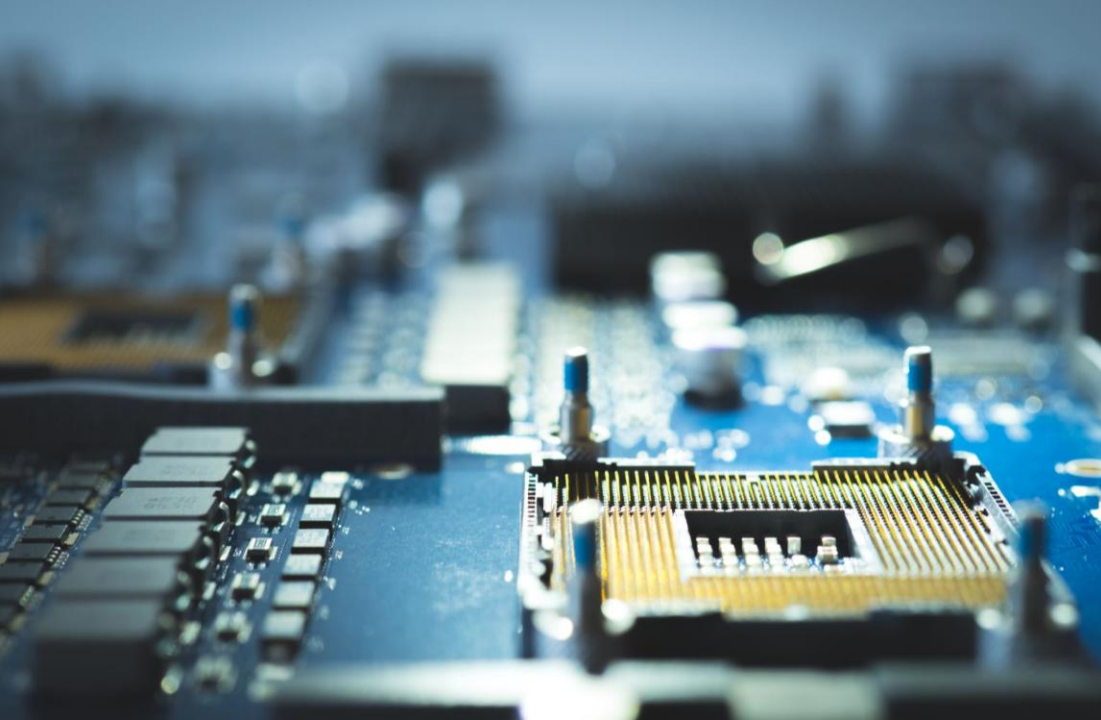
- Prioritize depthwise-separable convolutions, inverted residuals, and squeeze-and-excitation with careful channel scaling to reduce MACs and bandwidth.
 - Choose input resolutions and channel widths that fit on-chip memory to minimize DRAM traffic; cap peak activation sizes.
 - Constrain the operator set to what the device delegate accelerates (Hexagon/QNN, NNAPI, Core ML, CUDA/TensorRT) to avoid falling back to CPU.
 - Optimize for energy: use quantization (INT8), layer fusion, and DVFS-friendly throughput; target steady 20–30 fps at modest clocks rather than peak bursts.
-



Architecture Selection Principles

For Microcontrollers (Memory-Constrained)

- Design with fixed memory budgets: model parameters must fit in flash; the largest intermediate tensor set must fit in SRAM.
 - Use INT8 quantization end-to-end; prefer operator patterns available in micro inference libraries (e.g., TFLite Micro).
 - Minimize simultaneous live activations: structure blocks to reuse buffers and avoid long skip connections that force multiple feature maps to reside in SRAM.
 - Keep kernels small and channels modest; rely on pruning and knowledge distillation to recover accuracy within strict size limits.
-



Workflow

- Check supported precisions on the target.
- Start with INT8 (mobile), FP16/INT8 (server), tighter bits only if necessary (MCU).
- Calibrate, test on device, iterate.

Quantization Strategy Selection

Goal: Pick the lowest precision your hardware supports without breaking accuracy.

Server GPUs

- Train: FP32 or mixed FP16.
- Infer: FP16/TF32/INT8 on tensor cores.
- Use higher precision if accuracy is critical.

Mobile Processors

- Default to INT8 for weights and activations.
- Use per-channel weights; keep 32-bit accumulators.
- Verify kernels run on the device accelerator, not CPU.

Microcontrollers

- Go lower if needed: INT8 → 4-bit weights where supported.
- Consider ultra-low-bit activations only when memory forces it.
- Ensure weights fit in flash and peak activations in SRAM.
- Quantization-Aware Training
- Use QAT for aggressive quantization or MCU targets to recover accuracy.

Memory-Constrained Model Design

Goal: Fit models in limited flash/SRAM while keeping accuracy usable

Budget Planning

- Know flash size (weights/code) and SRAM size (activations/scratch)
- Track peak live activations, not just parameters
- Use static memory plans; preallocate reuse buffers

Design Techniques

- Layer-wise processing
- Run one layer at a time
- Stream weights from flash
- Reuse a single activation buffer across layers

In-place ops

- Use ReLU and similar in-place to avoid extra tensors
- Minimize temporary buffers

Operator fusion

- Fuse Conv + BN + ReLU
- Reduce intermediate tensors and memory traffic

Quantization

- INT8 weights/activations; 32-bit accumulators if needed

Pruning and sparsity

- Remove low-importance channels/connections
- Exploit sparse formats when supported

Shape control

- Cap feature-map sizes (channels/resolution)
- Avoid long skip connections that keep tensors live

Validation

- Measure peak SRAM during inference on device
- Iterate until within flash/SRAM limits

Hardware-Software Co-Design Methodology

Goal: Co-optimize model and platform for accuracy, latency, power, and cost

1. **Requirements:** **a)** Accuracy target, **b)** Latency budget, Power/thermal limits, **c)** Cost and BOM constraints
2. **Hardware Selection:** **a)** Compute type (CPU/GPU/NPU/DSP), **b)** Memory size and bandwidth, **c)** Power delivery/thermal headroom, **d)** Toolchain and software ecosystem
3. **Model Co-Optimization:** **a)** Shape architecture to hardware limits (ops, tensor sizes), **b)** Choose precision the hardware accelerates (FP16/INT8/4-bit), **c)** Layout tensors for fast access; fuse ops to cut memory traffic
4. **Software Optimization:** **a)** Use hardware-specific kernels/intrinsics, **b)** Optimize memory access (tiling, cache reuse, DMA), **c)** Exploit parallelism and pipelining; pin critical ops to accelerators

Case Study: Wake-Word on MCU (example outcomes)

- Target: <100 ms latency, <10 mW, <500 KB model
- Use depthwise/1×1 convs with DSP intrinsics
- Global average pooling to avoid large FC layers
- Small classifier that fits in fast memory
- INT8 quantization end-to-end

In-Class Exercise 2 (20 minutes)

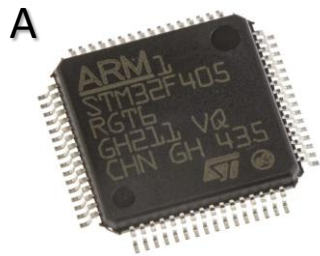
Hardware-Software Co-Design

Scenario: Industrial Predictive Maintenance

Application: Bearing fault detection in manufacturing equipment

- **Sensor:** Vibration sensor sampling at 10 kHz
- **Environment:** Factory floor (temperature, vibration, electromagnetic interference)
- **Constraints:** Battery-powered, 1-year operation, <\$20 cost

Hardware Options



ARM Cortex-M4:
DSP instructions,
512KB flash,
128KB RAM



ESP32: WiFi capable,
4MB flash, 320KB RAM



nRF52840: Bluetooth Low Energy,
1MB flash, 256KB RAM

Your Task (Groups of 3-4):

1. Select optimal hardware platform
2. Design model architecture considering:
 - Sensor data characteristics (vibration signals)
 - Memory constraints
 - Power requirements for 1-year operation
3. Plan quantization and optimization strategy
4. Estimate feasibility of meeting all requirements
5. Time: 10 minutes analysis + 10 minutes discussion

Energy-Aware Design Optimization

Minimizing Power Consumption

Power sources to watch

1. Computation energy

- 8-bit multiply $\approx$ 20 femtojoules
- 16-bit multiply $\approx$ 80 femtojoules
- 32-bit multiply $\approx$ 320 femtojoules
- Memory read/write: roughly 5–1000 picojoules depending on the level

2. Memory access energy

- Register access is cheapest
- Cache access costs a few to tens of picojoules
- DRAM access is most expensive: 1–10 nanojoules

3. Peripheral energy

- Sensor sampling: microjoules per sample
- Radio transmission: millijoules per packet

Optimization strategies

Reduce computation

- Prefer bit shifts and adds over multiplies when mathematically equivalent
- Use lower precision (int8/int16) where accuracy allows
- Fuse operations and reuse intermediate results

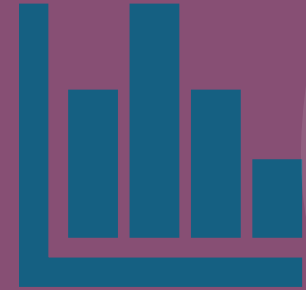
Minimize memory access

- Process data in place to avoid copies
- Keep hot data in registers or on-chip SRAM; avoid DRAM
- Arrange data for cache-friendly, sequential access
- Use fixed-size buffers and avoid dynamic allocation

Duty cycling

- Sleep between capture and inference cycles
- Batch work: wake briefly, process, then return to deep sleep
- Keep radios off; transmit only summaries or alerts at low duty cycle

Performance Validation and Benchmarking



Measuring real-world performance - Benchmarking methodology

1) Accuracy validation

- Compare the optimized model against a baseline on a representative test set. Report final accuracy and the drop from baseline. Target is minimal loss while gaining efficiency.

2) Performance measurement

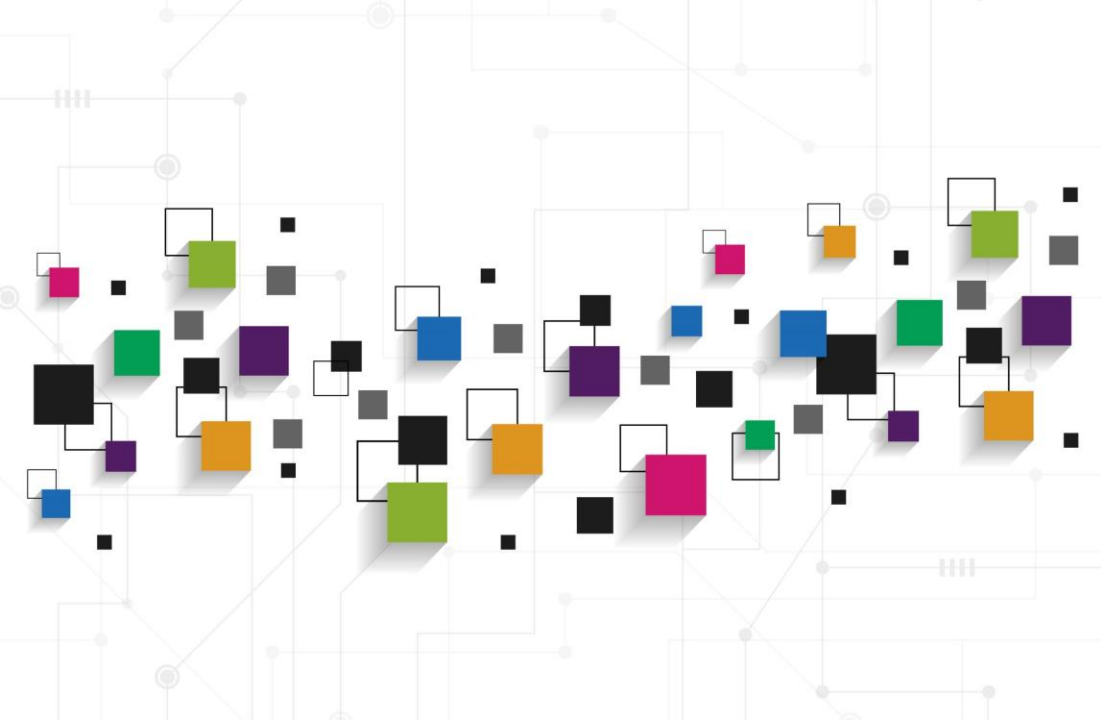
- Measure end-to-end inference time on the actual device. Use timestamps around the full pipeline and report average, p50, p90, and worst-case latency over many runs.

3) Energy measurement

- Profile current draw during active processing and during sleep. Multiply by supply voltage to get power, then integrate over time to estimate energy per inference and average current at your duty cycle. Include radio and sensor activity if used.

Validation criteria

- Accuracy: less than 2% absolute degradation from baseline
- Latency: meets real-time requirements with margin
- Energy: fits the battery-life budget for the intended duty cycle
- Memory: stays within RAM and flash limits with headroom



Managing Hardware Diversity - Deployment Challenges

Platform Fragmentation:

- Different instruction sets: ARM, x86, RISC-V
- Different accelerators: GPUs, NPUs, DSPs
- Different memory architectures: Cached vs. scratchpad
- Different software stacks: Operating systems, drivers, compilers

Cross-Platform Deployment Strategies

Abstraction Strategies

1. Framework-Level Abstraction

Same model, multiple backends
Auto-optimization per target

2. Compiler-Based Optimization

Compile once, retarget to CPUs/GPUs/MCUs

3. Hardware Abstraction Layer

Standardized ops isolated from device specifics

Benefits:

- Single codebase for multiple targets
- Automatic optimization for each platform
- Easier maintenance and updates

Future Directions in AI Hardware

Emerging Trends and Technologies

Emerging Architectures:

1. Processing-in-Memory (PIM)

- Computation directly in memory arrays
- Eliminates data movement for memory-bound operations
- Reduces energy consumption by orders of magnitude

2. Neuromorphic Computing

- Brain-inspired spiking neural networks
- Event-driven computation
- Ultra-low power for always-on applications

3. Optical Computing

- Light-based matrix multiplication
- Massive parallelism at speed of light
- Potential for exascale AI systems

4. Quantum-Inspired Algorithms

- Quantum annealing for optimization problems
- Hybrid classical-quantum systems
- New algorithmic approaches enabled by quantum properties

Hardware-Software Evolution:

- Compilation: Automatic optimization across entire stack
- Adaptivity: Runtime adaptation to changing workloads
- Efficiency: Continued focus on energy per operation
- Integration: Closer coupling between algorithm and hardware design

The Path Forward:

Hardware specialization will continue driven by the end of Moore's Law and increasing computational demands. Success requires co-design of algorithms, software, and hardware optimized for specific application domains.

Moore's Law is the observation that the number of transistors on a microchip roughly doubles every two years, leading to a continuous increase in computing power and decrease in cost.

Summary - Week 6 Key Takeaways

Hardware Specialization - Core Principles

- **Specialization Drives Efficiency:** Domain-specific hardware dramatically outperforms general-purpose processors for AI workloads
- **Memory is the Bottleneck:** Data movement costs dominate computation costs in modern AI systems
- **Precision Flexibility:** Trading numerical precision for efficiency enables deployment on resource-constrained hardware
- **Co-Design is Essential:** Optimal systems require joint optimization of algorithms, software, and hardware

Hardware Spectrum Understanding:

- **Datacenter:** Massive parallel processors (GPUs, TPUs) optimized for training throughput
- **Edge/Mobile:** Balance of performance and efficiency for real-time inference
- **Microcontroller:** Extreme efficiency optimization for always-on operation

Design Methodology:

- **Requirements-Driven:** Start with application constraints (power, cost, latency)
- **Hardware-Aware:** Design models that leverage target hardware capabilities
- **Validation-Focused:** Measure actual performance, not just theoretical metrics
- **Cross-Platform:** Plan for deployment across diverse hardware targets

Practical Impact:

- Enable AI deployment in resource-constrained environments
- Reduce energy consumption through specialized hardware
- Make edge computing practical for complex AI applications
- Bridge the gap between research models and deployed systems



Looking Ahead - Week 7 Preview

Machine Learning for Systems

Next Week's Focus:

- ML-driven system optimization across scales
- Adaptive systems and runtime optimization
- Energy-aware ML systems
- Feedback loops between ML predictions and system behavior

Connection to Hardware Specialization:

- Understanding hardware enables more effective ML for systems optimization
- Hardware performance models become inputs to ML-driven scheduling
- Specialized hardware creates new opportunities for adaptive systems
- Energy characteristics inform ML-based power management

Required Readings for Week 7:

- [Book 1 - Volume 2 — Performance Engineering](#)
 - [Book 1 - Volume 2 — Edge Intelligence](#)
 - [Book 1 - Volume 2 — Hardware Acceleration](#)
 - [Book 2: Chapter 15: Optimizing Latency](#)
 - [Book 2: Chapter 16: Optimizing Energy Usage](#)
- 

Programming Assignment 3: Hardware-Aware ML Design

Objective: Design & optimize a computer vision model for multiple hardware platforms

What You'll Build:

- Baseline MobileNetV2: Image classifier on CIFAR-10 (target >85% accuracy)
- 4 Optimization Variants: Latency-optimized, memory-optimized, energy-efficient, and quantized models
- Cross-Platform Analysis: Performance comparison across CPU, ARM, MCU, and mobile GPU platforms
- Design Framework: Methodology for systematic hardware-aware optimization decisions

Two Implementation Tracks:

- Track A - Real Hardware: Deploy on actual devices (Intel/ARM CPUs, mobile GPUs, edge AI boards)
- Track B - Simulation & Modeling: Use QEMU, Renode, WebGPU proxies (laptop-only, full credit available)

Key Learning Outcomes:

- Understand how hardware constraints drive ML architecture decisions
- Master trade-offs between accuracy, inference speed, memory usage, and energy consumption
- Develop platform-specific optimization strategies for edge deployment

Deliverables: Complete codebase, trained models, performance benchmarks, 3-4 page analysis report

Due on: Oct 27, 23:59.

Midterm Exam

- **Date:** Tuesday, 13 October 2026
- **Time:** 19:00–20:00
- **Venue:** Room 3, 30/F, Tower 1, The Millennity, Kwun Tong
- **Format:** Paper and pen
- **External resources are not allowed.**
- **Please bring your HKID card for verification.**
- **The class will continue after the exam, so you may bring your laptop. However, laptops may not be opened or used during the exam.**